Arduino Alvik API Overview

A technical summary of the APIs used to control the Alvik Robot.

Author · Paolo Cavagnolo Last revision · 09/30/2024

API List

To access to any of these functions you need first to initialize an instance of the class **ArduinoAlvik()**. This reference is useful for both **MicroPython** and **C++ environments** as the functions were all created to have the same form across development environments meaning your experience should be easy to carry both options.

```
alvik = ArduinoAlvik()
```

Then you can use the following functions as methods of the instance you've created, for example:

```
alvik.begin()
```

`is_on`

Returns true if robot is on

Outputs

boolean: Returns true if robot is on, false if it is off.

`begin`

begin()

Begins all Alvik operations

`is_target_reached`

is_target_reached()

Returns True if robot has sent an M or R acknowledgment. It also responds with an ack received message

Outputs

boolean: Returns True if robot has arrived to target, False otherwise.

`stop`

stop()

Stops all Alvik operations

`get_orientation`

get_orientation()

Returns the orientation of the IMU

Outputs

r: roll value

p: pitch value

y: yaw value

`get_accelerations`

get_accelerations()

Returns the 3-axial acceleration values of the IMU

Outputs

- **ax**: acceleration on x
- **ay**: acceleration on y
- **az**: acceleration on z

`get_gyros`

get_gyros()

Returns the 3-axial angular acceleration of the IMU

Outputs

- **gx**: angular acceleration on x
- **gy**: angular acceleration on y
- **gz**: angular acceleration on z

`get_imu`

get_imu()

Returns all the IMUs readouts

Outputs

- **ax**: acceleration on x
- **ay**: acceleration on y
- **az**: acceleration on z
- **gx**: angular acceleration on x
- **gy**: angular acceleration on y
- **gz**: angular acceleration on z

`get_line_sensors`

get_line_sensors()

Returns the line follower sensors readout

Outputs

- **left**: left sensor readout

`brake`

brake()

Brakes the robot

`get_ack`

get_ack()

Returns last acknowledgement

Outputs

last_ack: last acknowledgement value.

`get_battery_charge`

get_battery_charge()

Returns the battery SOC

Outputs

battery_soc: percentage of charge.

`get_touch_any`

get_touch_any()

Returns true if any button is pressed

Outputs

touch_any: true if any button is pressed, false otherwise.

`get_touch_ok`

get_touch_ok()

Returns true if ok button is pressed

Outputs

touch_ok: true if ok button is

`get_touch_cancel`

get_touch_cancel()

Returns true if cancel button is pressed

Outputs

touch_cancel: true if cancel button is pressed, false otherwise.

`get_touch_center`

get_touch_center()

Returns true if center button is pressed

Outputs

touch_center: true if center button is pressed, false otherwise.

`get_touch_up`

get_touch_up()

Returns true if up button is pressed

Outputs

touch_up: true if up button is pressed, false otherwise.

`get_touch_left`

get_touch_left()

Returns true if left button is pressed

Outputs

touch_left: true if left button is pressed, false otherwise.

`get_touch_down`

Returns true if down button is pressed

Outputs

touch_down: true if down button is pressed, false otherwise.

`get_touch_right`

get_touch_right()

Returns true if right button is pressed

Outputs

touch_right: true if right button is pressed, false otherwise.

`get_color_raw`

get_color_raw()

Returns the color sensor's raw readout

Outputs

color: the color sensor's raw readout

`get_color_label`

get_color_label()

Returns the label of the color as recognized by the sensor

Outputs

color: the label of the color as recognized by the sensor

`get_version`

get_version()

Returns the firmware version of the

version: Returns the firmware version of the Alvik

`print_status`

print_status()

Prints the Alvik status

Outputs

status: Prints the Alvik status

`set_behaviour`

set_behaviour(behaviour: int)

Sets the behaviour of Alvik

Inputs

behaviour: behaviour code

`rotate`

rotate(angle: float, unit: str = 'deg', blocking: bool = True)

Rotates the robot by given angle

Inputs

angle: the angle value

unit: angle unit

blocking:*True* or *False*

`move`

move(distance: float, unit: str = 'cm', blocking: bool = True)

Moves the robot by given distance

Inputs

distance: the distance value

unit: the distance unit. ?

`get_wheels_speed`

get_wheels_speed(unit: str = 'rpm')

Inputs

unit: unit of rotational speed of the wheels. ?

Outputs

left_wheel_speed: the speed value

right_wheel_speed: the speed value

Returns the speed of the wheels

`set_wheels_speed`

set_wheels_speed(left_speed: float, right_speed: float, unit: str = 'rpm')

Sets left/right motor speed

Inputs

left_speed: the speed value

right_speed: the speed value

unit: unit of rotational speed of the wheels. ?

`set_wheels_position`

set_wheels_position(left_angle: float, right_angle: float, unit: str = 'deg')

Sets left/right motor angle

Inputs

left_angle: the angle value

right_angle: the angle value

unit: the angle unit, ?

`get_wheels_position`

Returns the angle of the wheels

Inputs

angular_unit: unit of rotational speed of the wheels. ?

Outputs

angular_velocity: speed of the wheels.

`drive`

drive(linear_velocity: float, angular_velocity: float, linear_unit: str = 'cm/s',angular_unit: str = 'deg/s')

Drives the robot by linear and angular velocity

Inputs

linear_velocity: speed of the robot.

angular_velocity: speed of the wheels.

linear_unit: unit of linear velocity. ?

angular_unit: unit of rotational speed of the wheels. ?

`get_drive_speed`

get_drive_speed(linear_unit: str = 'cm/s', angular_unit: str = 'deg/s')

Returns linear and angular velocity of the robot

Inputs

linear_unit: unit of linear velocity. ?

angular_unit: unit of rotational speed of the

Outputs

- **linear_velocity**: speed of the robot.
- **angular_velocity**: speed of the wheels.

`reset_pose`

reset_pose(x: float, y: float, theta: float, distance_unit: str = 'cm', angle_unit: str = 'deg')

Resets the robot pose

Inputs

- **x**
- **y**
- **theta**
- **distance_unit**: unit of x and y outputs, ?
- **angle_unit**: unit of theta output, ?

`get_pose`

get_pose(distance_unit: str = 'cm', angle_unit: str = 'deg')

Returns the current pose of the robot

Inputs

- **distance_unit**: unit of x and y outputs, ?
- **angle_unit**: unit of theta output, ?

Outputs

- **x**
- **y**
- **theta**

`set_servo_positions`

Sets A/B servomotor angle

Inputs

a_position: position of A servomotor (0-180)

b_position: position of B servomotor (0-180)

`set_builtin_led`

set_builtin_led(value: bool)

Turns on/off the builtin led

Inputs

value: True = ON, False = OFF

`set_illuminator`

set_illuminator(value: bool)

Turns on/off the illuminator led

Inputs

value: True = ON, False = OFF

`color_calibration`

color_calibration(background: str = 'white')

Calibrates the color sensor

Inputs

background: string "white" or "black"

`rgb2hsv`

rgb2hsv(r: float, g: float, b: float)

Converts normalized rgb to hsv

Inputs

b: blue value

Outputs

h: hue value

s: saturation value

v: brightness value

`get_color`

get_color(color_format: str = 'rgb')

Returns the normalized color readout of the color sensor

Inputs

color_format: rgb or hsv only

Outputs

r or **h**

g or **s**

b or **v**

`hsv2label`

hsv2label(h, s, v)

Returns the color label corresponding to the given normalized HSV color input

Inputs

h: hue value

s: saturation value

v: brightness value

Outputs

color label: like "BLACK" or "GREEN", if possible, otherwise return "UNDEFINED"

`get_distance`

Returns the distance readout of the TOF sensor

Inputs

unit: distance output unit

Outputs

left_tof: 45° to the left object distance

center_left_tof: 22° to the left object distance

center_tof: center object distance

center_right_tof: 22° to the right object distance

right_tof: 45° to the right object distance

`get_distance_top`

get_distance_top(unit: str = 'cm')

Returns the obstacle top distance readout

Inputs

unit: distance output unit

Outputs

top_tof: 45° to the top object distance

`get_distance_bottom`

get_distance_bottom(unit: str = 'cm')

Returns the obstacle bottom distance readout

Inputs

unit: distance output unit

Outputs

bottom_tof: 45° to the

`on_touch_ok_pressed`

on_touch_ok_pressed(callback: callable, args: tuple = ())

Register callback when touch button OK is pressed

Inputs

callback: the name of the function to recall

args: optional arguments of the function

`on_touch_cancel_pressed`

on_touch_cancel_pressed(callback: callable, args: tuple = ())

Register callback when touch button CANCEL is pressed

Inputs

callback: the name of the function to recall

args: optional arguments of the function

`on_touch_center_pressed`

on_touch_center_pressed(callback: callable, args: tuple = ())

Register callback when touch button CENTER is pressed

Inputs

callback: the name of the function to recall

args: optional arguments of the function

`on_touch_up_pressed`

on_touch_up_pressed(callback:

Register callback when touch button UP is pressed

Inputs

callback: the name of the function to recall

args: optional arguments of the function

`on_touch_left_pressed`

on_touch_left_pressed(callback: callable, args: tuple = ())

Register callback when touch button LEFT is pressed

Inputs

callback: the name of the function to recall

args: optional arguments of the function

`on_touch_down_pressed`

on_touch_down_pressed(callback: callable, args: tuple = ())

Register callback when touch button DOWN is pressed

Inputs

callback: the name of the function to recall

args: optional arguments of the function

`on_touch_right_pressed`

on_touch_right_pressed(callback: callable, args: tuple = ())

Register callback when touch button RIGHT is pressed

Inputs

callback: the name of the function to recall

args: optional arguments of the function

Function Parameters

The Distance Unit

Distance unit of measurement used in the APIs:

cm: centimeters

mm: millimeters

m: meters

inch: inch, 2.54 cm

in: inch, 2.54 cm

The Angle Unit

Angle unit of measurement used in the APIs:

deg: degrees, example: 1.0 as reference for the other unit. 1 degree is 1/360 of a circle.

rad: radiant, example: 1 radiant is 180/pi deg.

rev: revolution, example: 1 rev is 360 deg.

revolution: same as rev

perc: percentage, example 1 perc is 3.6 deg.

%: same as perc

The Linear Speed Unit

Speed unit of measurement used in the APIs:

'cm/s': centimeters per second

'mm/s': millimeters per second

'm/s': meters per second

'in/s': inch per second

The Rotational Speed Unit

Rotational speed unit of measurement used in the APIs:

'rpm': revolutions per minute, example: 1.0 as reference for the other unit.

'deg/s': degrees per second, example: 1.0 deg/s is 60.0 deg/min that is 1/6 rpm.

'rad/s': radiant per second, example: 1.0 rad/s is 60.0 rad/min that is 9.55 rpm.

'rev/s': revolution per second, example: 1.0 rev/s is 60.0 rev/min that is 60.0 rpm.

`"blocking" or "non blocking"`

While programming a microcontroller, the term "blocking" means that **all the resources are used only in performing a specific action, and no other things can happen at the same time**. Usually this is used when you want to be precise or you don't want anything else that could interact with the action you are performing.

On the other hand, "Non blocking", means that the microcontroller is free to do other things while the action is been performed.

Examples

These examples demonstrate practical implementations on how to use the Arduino Alvik API. Whether you're working with MicroPython or C++, these simple

in your projects, making it easy to get started with the Alvik in the language you're most comfortable with.

Simple Color Sensing Example

This example demonstrates how to implement basic color sensing. The Alvik's color sensor reads the color of an object placed under it, and the detected color is printed to the console.

Reference for MicroPython

```
from arduino_alvik impor
from time import sleep_m

alvik = ArduinoAlvik()
alvik.begin()

print("Alvik initialized

while True:
    color = alvik.get_co
    print(f"Detected col
    sleep_ms(500)
```

Reference for C++

```
#include "Arduino_Alvik.

Arduino_Alvik alvik;

void setup() {
  Serial.begin(9600);
  alvik.begin();
  Serial.println("Alvik
}

void loop() {
  char* color = alvik.ge
  Serial.print("Detected
  Serial.println(color);
  delay(500);
}
```

Simple Directional Control

This example demonstrates very basic control over the robot's movement based on directional arrow buttons. The Alvik will drive in the direction corresponding to the arrow button pressed (up, down, left, right).

Reference for MicroPython

```
from arduino_alvik impo
from time import sleep_

alvik = ArduinoAlvik()
alvik.begin()

print("Alvik initialize

while True:
    if alvik.get_touch_
        print("Moving l
        alvik.drive(10
        sleep_ms(1000)
        alvik.brake()
    elif alvik.get_touc
        print("Moving D
        alvik.drive(-10
        sleep_ms(1000)
        alvik.brake()
    elif alvik.get_touc
        print("Turning
        alvik.drive(0,
        sleep_ms(1000)
        alvik.brake()
    elif alvik.get_touc
        print("Turning
        alvik.drive(0,
        sleep_ms(1000)
```

Reference for C++

```
#include "Arduino_Alvi

Arduino_Alvik alvik;

void setup() {
  Serial.begin(9600);
  alvik.begin();
  Serial.println("Alvi
}

void loop() {
  if (alvik.get_touch_u
    Serial.println("Mov
    alvik.drive(100, 0,
    delay(1000);
    alvik.brake();
  } else if (alvik.get_
    Serial.println("Mov
    alvik.drive(-100, 0
    delay(1000);
    alvik.brake();
  } else if (alvik.get_
    Serial.println("Tur
    alvik.drive(0, 100,
    delay(1000);
    alvik.brake();
  } else if (alvik.get_
    Serial.println("Tur
```

Line Following

This example demonstrates how to create a simple line-following robot. The code initializes the Alvik, reads sensor data to detect the line, calculates the error from the center of the line, and adjusts the Alvik's wheel speeds to follow the line. It also uses LEDs to indicate the direction the robot is turning.

The Alvik starts when the OK ✓ button is pressed and stops when the Cancel button is pressed. The Alvik continuously reads the line sensors, calculates the error, and adjusts the wheel speeds to correct its path.

```
from arduino_alvik impo
from time import sleep_
import sys


def calculate_center(le
    centroid = 0
    sum_weight = left +
    sum_values = left +
    if sum_weight != 0
        centroid = sum_
        centroid = 2 -
    return centroid


alvik = ArduinoAlvik()
alvik.begin()

error = 0
control = 0
kp = 50.0

alvik.left_led.set_colo
alvik.right_led.set_co

while alvik.get_touch_c
    sleep_ms(50)

```

Reference for C++

```
/*
    This file is part c

    Copyright (c) 2024

    This Source Code Fc
    License, v. 2.0. It
    file, You can obta:

*/

#include "Arduino_Alvi

Arduino_Alvik alvik;

int line_sensors[3];
float error = 0;
float control = 0;
float kp = 50.0;



void setup() {
  Serial.begin(115200)
  while((!Serial)&&(mi
  alvik.begin();
  alvik.left_led.set_c
  alvik.right_led.set_
```